

ABSTRACT

The Devanagari Character Recognition System is designed to perform Optical Character Recognition (OCR) on Devanagari characters, supporting a wide range of Nepali alphabets. The system architecture comprises five main stages: preprocessing, segmentation, feature extraction, prediction, and postprocessing. The solution leverages a Convolutional Neural Network (CNN) for model training and image processing techniques for accurate character recognition. The backend is developed using Flask, providing API and business logic functionalities, while the frontend is implemented in React for an interactive user experience. PostgreSQL serves as the database for secure user authentication, image storage, and maintaining a prediction history for future reference.

The system allows users to upload images for recognition without quality loss and includes an audio generation feature for recognized characters. Images are also stored for future model training to improve accuracy. Extensive testing of different algorithms revealed varying levels of accuracy, emphasizing the system's adaptability. The project demonstrates a collaborative effort integrating software and hardware components, offering a robust solution for recognizing Devanagari characters.

Keywords: Devanagari character recognition, Convolutional Neural Network, image processing, optical character recognition, user authentication, image segmentation.

Contents

ABSTRACT	i
List of Figures	iv
List of Abbreviations.....	v
Chapter 1: Introduction	1
1.1 Introduction	1
1.1.1 Devanagari Characters	1
1.1.2 Optical Character Recognition.....	1
1.1.3 Machine Learning	1
1.1.4 Deep Learning.....	2
1.1.5 Convolutional Neural Networks (CNN)	3
1.2 Problem Definition	4
1.3 Objectives	4
1.4 Scope and Limitation.....	5
1.5 Significance of the Project.....	5
1.6 Features of Project.....	5
1.7 Report Organization	6
Chapter 2: Literature Review	7
2.1 Review of Papers.....	7
2.1.1 Offline Handwritten Numerals Recognition using Correlation	7
2.1.2 Devanagari Character Recognition Using Neural Networks	7
2.1.3 Handwritten Devanagari Character Recognition using Neural Network	8
Chapter 3: System Analysis	9
3.1 Requirement Analysis	9
3.1.1 Functional Requirements	9
3.1.2 Non-Functional Requirements	10
3.2 Feasibility Analysis	11
3.2.1 Economic Feasibility	11
3.2.2 Operational Feasibility.....	11
3.2.3 Technical Feasibility	11
3.2.4 Schedule Feasibility	12
3.3 Constructing System Requirements	12
Chapter 4: System Design and Implementation.....	14
4.1 System Design	14
4.1.1 Block Diagram.....	14
4.1.2 Dataset Preparation	15

4.2 Implementation.....	15
4.2.1 Implementation in Python.....	15
4.3 Neural Network Design and Implementation.....	16
4.3.1 Convolutional Neural Network.....	16
4.4 Image Processing Model	17
4.4.1 Image Acquisition.....	17
4.4.2 Image Preprocessing	18
4.4.4 Character Prediction/classification	19
4.4.5 Character Localization.....	19
4.4.6 Character Recognition	20
4.5 List of Algorithms	20
4.5.1 Load Dataset	20
4.5.2 Create CNN Model	20
4.5.3 Train CNN Model.....	21
4.5.4 Image Processing	21
4.5.5 Character Recognition & Prediction.....	21
4.5 Usage of Algorithms in Our Project	22
Chapter 5: System Testing	23
5.1 Unit Testing.....	23
5.2 Integration Testing.....	26
5.3 System Testing	28
Chapter 6: Maintenance and Support	29
6.1 Maintenance	29
6.2 Support	29
Chapter 7: Conclusion & Future enhancements.....	30
7.1 Conclusion.....	30
7.2 Future enhancements	30
Bibliography.....	31
APPENDIX	32
Appendix A: Source Code	32
i. Model Training	32
ii. Image Processing	33
iii. RESTful APIs	38

List of Figures

Figure 1. 1: CNN	3
Figure 3. 1: Use Case Diagram	9
Figure 3. 2: Gantt Chart.....	12
Figure 3. 3: DFD	13
Figure 4. 1: Block Diagram of DCR system	14
Figure 4. 7: CNN Architecture	16
Figure 4. 14: Character Creation using paint	18
Figure 5. 1.1: Login screen.....	23
Figure 5.1.2: Login success	24
Figure 5.1.3: Model Prediction Integration	25
Figure 5.2.1: Integration Testing of File Upload.....	26
Figure 5.2.2: API Testing using Postman.....	27

List of Abbreviations

DCR	Devanagari Character Recognition
NN	Neural Network
CNN	Convolutional Neural Network
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
FFNN	Feed Forward Neural Network
RELU	Rectified Linear Unit
ML	Machine Learning
DL	Deep Learning
AI	Artificial Intelligence
OCR	Optical Character Recognition
IDLE	Integrated Development Environment
CSV	Comma Separated Value

Chapter 1: Introduction

1.1 Introduction

1.1.1 Devanagari Characters

The Devanagari script, which is Nepal's national font and is widely used across India, encompasses a comprehensive character set including 10 numerals, 36 consonants, and 13 vowels. The subtle structural similarities between certain characters make this script particularly interesting from a recognition perspective. For instance, distinguishing between 'ब' and 'व' requires identifying a minute difference in the internal cross mark, while 'ड' and 'ढ' are differentiated solely by the presence of a dot. Similar structural patterns are observed between characters like 'घ', 'ग', and 'ङ', as well as between the numeral '२' and the consonant 'र'. The script also features compound characters such as 'क्ष', 'त्र', and 'ज्ञ', which are formed by combining basic consonants, adding another layer of complexity to character recognition. These intricate similarities and compound formations present unique challenges in developing accurate recognition systems, requiring sophisticated image processing and pattern recognition techniques to achieve reliable character identification.

1.1.2 Optical Character Recognition

Through the years, character recognition technology has advanced dramatically and become an essential tool for a wide range of applications. Optical Character identification (OCR) systems are used in many aspects of modern computing, from aiding robotic vision systems to translating documents to digital text and facilitating automatic number plate identification. These recognition systems fall into two primary categories: offline character recognition, which uses previously typed or printed text, and online character recognition, which processes characters as they are written in real time. Since it may operate independently after the OCR model has been trained, the latter has become especially well-known for its adaptability and broader applicability. Machine learning technology, which is at the heart of these systems, allows computers to learn and increase the accuracy of their recognition through pattern analysis and deep learning algorithms, boosting their dependability for real-world applications in both business and daily life.

1.1.3 Machine Learning

Machine Learning is a breakthrough paradigm in computer science, enabling systems to acquire information through pattern recognition rather than traditional programming methods. As a core component of artificial intelligence, it relies on data-driven principles, where systems acquire knowledge through exposure to training instances. In our Devanagari Character Recognition project, this means the algorithm learns to identify characters not through predefined rules, but by examining multiple handwritten samples, adapting to varied writing styles and variances. The applications of ML extend far beyond character recognition, embracing fields like email filtering and computer vision, where conventional programming approaches would be inadequate. What makes ML so powerful is its capacity to boost performance through continual learning, especially helpful in tough jobs like discriminating between similar Devanagari letters. This adaptive learning power makes ML

a great choice for complicated pattern recognition issues where subtle distinctions, such as those seen in Devanagari script, need to be reliably discovered and processed. Some popular terminology used around ML environments are:

1. **Supervised Learning:** The program develops a mathematical model using a dataset that comprises both the inputs and the matching desired outcomes. For instance, when the aim is to detect whether an image contains a specific object, the training dataset for a supervised learning system would consist of photos that feature that object and images that do not (the inputs). Each image would be tagged (the outputs) to show if the object is present.
2. **Unsupervised Learning:** The algorithm effectively constructs a mathematical model from a dataset comprised solely of inputs, without any desired output labels. Unsupervised learning algorithms adeptly uncover structures within the data, facilitating the grouping and clustering of data points. This powerful approach reveals significant patterns and categorizes inputs through feature learning. Furthermore, dimensionality reduction decisively streamlines the dataset by reducing the number of "features" or inputs.
3. **Optimization:** The most significant element in the recipe for machine learning algorithms is the optimization process. It begins by creating a loss function or cost function and culminates in minimizing that loss using various optimization techniques. The choice of optimization technique can considerably effect whether one gets good accuracy in hours or days. The applications of optimization are boundless and are a widely investigated issue in both industry and academia. Common examples are SGD (Stochastic Gradient Descent), ADAM (Adaptive Moment Estimation), and Gradient Descent, among others.
4. **Loss:** A loss function is a non-negative value, and as this value drops, the model's resilience tends to improve. Some often used loss functions are Mean Squared Error (MSE) and Categorical Cross Entropy, among others.
5. **Underfitting and Overfitting:** Underfitting occurs when the training dataset is insufficient, leading to a high loss when evaluated against the test set. On the other hand, overfitting happens when the model is trained on a limited dataset for an extended period, causing it to memorize the training data instead of learning general patterns. Both issues result in elevated loss and reduced accuracy. Applying generalization techniques, selecting an appropriate loss function, and adjusting the training data and learning rate can help address these challenges.

1.1.4 Deep Learning

Deep learning represents an advanced subset of machine learning within artificial intelligence, differentiated by its capacity to interpret and learn from unstructured and unlabeled data without human supervision. At its base are neural networks that imitate the human brain's structure and function, with Convolutional Neural Networks (CNNs) playing a particularly vital role in image-processing tasks. In our Devanagari Character Recognition system, deep learning enables the model to automatically detect detailed patterns in character

images using numerous layers of neural networks. These networks progressively learn information from simple edges and curves to complicated character shapes, making them exceptionally good at managing the intricate distinctions in Devanagari characters.

1.1.5 Convolutional Neural Networks (CNN)

Convolutional neural networks (CNNs) are a sort of deep learning model frequently applied for tasks such as image categorization, similarity-based clustering (facilitating photo searches), and object detection in images. These advanced algorithms can identify various elements within visual data, ranging from faces and individuals to street signs and different species, such as platypuses.

In addition to image-related tasks, CNNs excel in optical character recognition (OCR), which helps convert printed or handwritten text into digital form. This process enables natural language processing for analyzing analog documents. Moreover, CNNs can also be employed in audio analysis when sound is represented visually as spectrograms. Recently, researchers have begun applying convolutional networks to text analysis and graph data using specialized techniques known as graph convolutional networks.

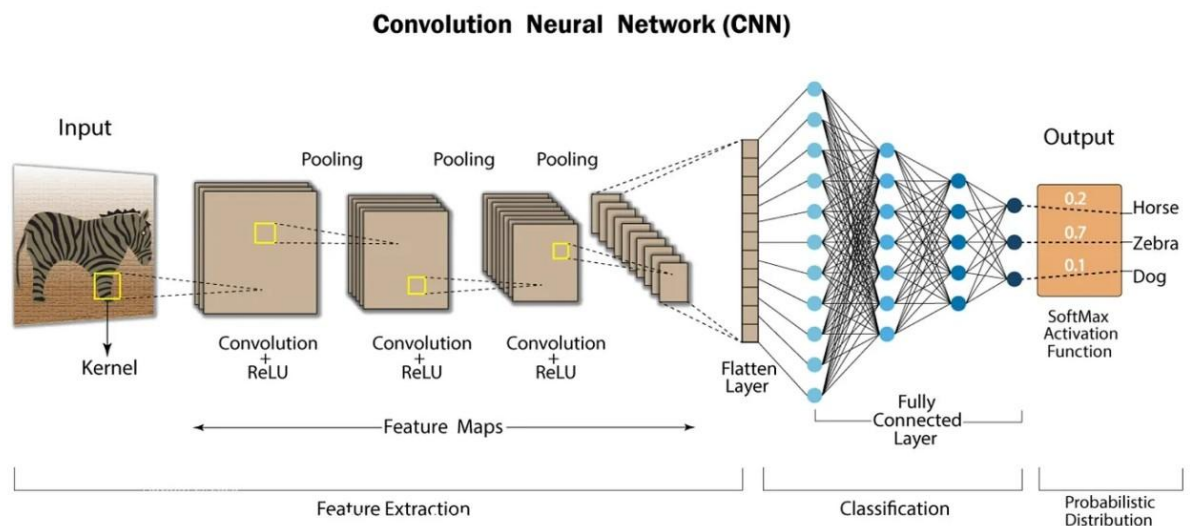


Figure 1. 1: CNN

The effectiveness of convolutional neural networks (CNNs) in recognizing images is a major factor behind the growing appreciation for deep learning. These networks are leading to groundbreaking developments in computer vision (CV), with important implications for various fields, including autonomous vehicles, robotics, drones, security systems, medical diagnostics, and interventions for individuals with vision impairments.

CNNs often employ the following layers:

1. **Convolutional layer** is the layer in which convolution takes place. Here, convolution is comparable to image processing. A filter of similar row and column or square size is taken and multiplied across the window that fits filter. The element-wise product is done and summed all. We commonly utilize stride, as how much pixel shift after executing one convolution. Also, zero padding is done sometime to add zeros. The convolution layer gives number of filters with same features. Here greater the number of filters, more the accurate model can get but computational complexity grows.

2. **Max-Pooling layer** is a layer where we grab only few pixels from preceding levels. We must specify a pool size and then that pool size is used on input pixels. The pool window is shifted across the entire input, and the maximum value within the overlapping input is selected. A pool of size (2, 2) outputs fifty percent of the input data.
3. **Dropout** is an essential technique in neural networks that helps prevent overfitting by randomly disconnecting neurons during training. It works like a filter that temporarily turns off a portion of connections between layers - for example, a dropout value of 0.5 means half of these connections are disabled during each training round. This random deactivation prevents the network from becoming too dependent on specific patterns or memorizing the training data exactly. Instead, it forces the network to learn more robust and generalized features, making it better at recognizing variations in input data, such as different handwriting styles in our Devanagari character recognition system. By introducing this controlled form of noise during training, dropout helps create a more reliable and adaptable model.
4. **Flatten layer** is one where multiple sized input is converted into 1d vector.
5. **Dense layer** in a convolutional neural network (CNN) is typically utilized for categorization following the convolution operations..

1.2 Problem Definition

Character recognition systems, while advanced for English and numerical scripts, face unique hurdles when processing Devanagari text. The complexity of Devanagari writing, with its similar-looking characters (like 'ब' and 'व'), compound characters (like 'क्ष', 'त्र', 'ज्ञ'), and intricate formation rules, makes accurate recognition difficult. This limitation affects different uses, from document digitization to number plate scanning. Our project aims to address these challenges by creating an efficient Devanagari Character Recognition system using modern machine learning techniques to improve accuracy in character identification.

1.3 Objectives

The primary objective of this system is to conduct extensive research on Devanagari character recognition in the field of Computer Vision. The system leverages a Convolutional Neural Network (CNN) model to provide reliable predictions for Devanagari characters. The system aims to offer a user-friendly and efficient solution for character recognition by utilizing simple image processing techniques.

Key objective include:

- Recognize and predict Devanagari characters from user-uploaded images stored locally, ensuring high accuracy with minimal quality loss.

1.4 Scope and Limitation

This project focuses on using Deep Learning techniques, specifically Flask and React-based web application for Devanagari character recognition. It employs image processing with OpenCV and CNN models to analyze and predict characters from uploaded images with user management capabilities.

Scope

- Recognizes 36 distinct Devanagari characters using CNN model integration
- Built with Flask backend for API handling and React frontend for user interface
- Implements JWT authentication and SQLAlchemy for data persistence
- Provides audio feedback for recognized characters using text-to-speech

Limitations

- Recognition limited to single character processing at a time
- Browser-based processing with memory constraints
- Model performance dependent on image quality and segmentation
- User interface restricted to web browser environment

Technical Stack

- Backend: Flask, OpenCV, SQLAlchemy, JWT
- Frontend: React, Vite, Tailwind CSS
- Database: PostgreSQL for user Management
- Processing: CNN model with 36-class prediction capability

The system aims to provide accurate character recognition while maintaining user data security and session management. The architecture follows standard software development practices with documented APIs and error handling mechanisms.

1.5 Significance of the Project

Devanagari character recognition through web application systems represents an innovative integration of modern software architecture and AI capabilities. This project implements a full-stack approach using Flask and React frameworks, combining RESTful APIs, image processing with OpenCV, and a 36-class CNN model for accurate character classification. The system's architecture incorporates JWT authentication, PostgreSQL database with SQLAlchemy ORM, and real-time prediction with confidence scoring and audio feedback. By leveraging these technologies alongside secure user management and efficient data tracking, the project demonstrates how modern web technologies can create robust character recognition systems. The implementation establishes a production-ready foundation that balances security, scalability, and user experience, contributing to the advancement of web-based AI applications for Devanagari script recognition while maintaining industry development standards.

1.6 Features of Project

The system implements a Flask-React web application that recognizes Devanagari characters through image uploads. It processes images using OpenCV and CNN models via authenticated APIs, providing predictions and audio feedback. The PostgreSQL database tracks user interactions and predictions while the React frontend ensures responsive user experience, effectively simplifying complex character recognition tasks through a modern web architecture.

1.7 Report Organization

In Chapter 1, this report introduces why the system is built and how. It also explains the objectives and features of the project in detail, along with the scope and limitations of the system.

In Chapter 2, this report discusses the existing systems related to Devanagari Character Recognition and reviews previous research and methodologies.

In Chapter 3, this report discusses the requirement analysis and feasibility study of the system. The process modeling techniques are used to provide insights into system requirements.

In Chapter 4, this report discusses the system design and implementation. It also covers the Neural Network Design, including dataset preparation, preprocessing, feature extraction, and Convolutional Neural Network (CNN) implementation.

In Chapter 5, this report discusses the tools used in this project to make it functional. The system testing process is explained in detail, including unit testing, integration testing, and system testing with structured input and output validation.

In Chapter 6, this report discusses the maintenance and support strategies implemented for the system, ensuring its long-term usability and performance improvements.

In Chapter 7, this report discusses the conclusion and potential future enhancements of the project, such as mobile application integration, real-time recognition, and advanced model optimizations.

Chapter 2: Literature Review

The purpose of this system is to recognize Devanagari characters through a modern web-based architecture. While existing solutions focus on standalone applications, this project implements a comprehensive Flask-React approach with real-time processing capabilities. The system integrates OpenCV preprocessing and CNN models within a production-grade web environment, addressing character recognition through authenticated APIs and database persistence.

By developing a full-stack solution with Flask backend and React frontend, this project offers unique insights into combining AI technologies with modern web frameworks. The system processes uploaded images through a secure pipeline, utilizing JWT authentication and PostgreSQL data management to maintain user interactions and prediction history. This implementation demonstrates practical application of character recognition within a scalable web architecture.

The system achieves accurate character prediction through its 36-class CNN model integration, complemented by confidence scoring and audio feedback features. While traditional approaches focus solely on recognition algorithms, this project emphasizes user experience through responsive design and intuitive interfaces. The implementation serves as a reference for developing production-ready AI applications within contemporary web frameworks, particularly for Devanagari script recognition tasks.

2.1 Review of Papers

2.1.1 Offline Handwritten Numerals Recognition using Correlation

In this paper, the author proposed a system designed to efficiently recognize offline handwritten digits with higher accuracy than previous methods. Previous handwritten number recognition systems were limited to recognizing only single digits, and they lacked the capability to recognize multiple numbers simultaneously. The author addressed this limitation by focusing on improving the segmentation process, which efficiently isolates individual digits. This enhancement allowed the system to perform better in terms of both accuracy and functionality for recognizing handwritten numerals.

2.1.2 Devanagari Character Recognition Using Neural Networks

In this paper, a neural network-based approach is proposed for building an automatic offline character recognition system for Devanagari. Devanagari is an Indo-Aryan script used predominantly in languages like Hindi and Marathi, with about 71 million speakers. While there has been considerable work done on character recognition for other Indian languages such as Hindi, Tamil, and Bengali, Devanagari has not seen much research, especially in character recognition. This paper addresses that gap by using a multilayer perceptron with a hidden layer for Devanagari character recognition. The characters are represented as patterns in a binary matrix format ($n \times n$) and stored in a file. A backpropagation neural network is employed for efficient recognition, and the rectified neuron values are transmitted through the feed-forward method in the neural network.

2.1.3 Handwritten Devanagari Character Recognition using Neural Network

In this digital era, organizations that store information in handwritten documents can utilize handwritten character recognition (HCR) to convert this information into digital form. Recognizing handwritten Devanagari characters presents challenges due to the presence of header lines, conjunct characters, and the similarity in shapes of multiple characters. This paper presents a grid-based method that combines image centroid zone and zone centroid zone techniques to recognize individual characters or numerical images. In the feature extraction process, the image is divided into n equal-sized grids or zones. The average distance of all pixels from the image centroid or the grid centroid is computed. The method combines the image centroid and zone centroid approaches to compute the average distance of all pixels in each grid with respect to both centroids, generating a feature vector of size $2 \times n$. This feature vector is then used as input to a feed-forward neural network for character recognition. The entire recognition process involves stages such as document preprocessing, segmentation, feature extraction using the grid-based approach, and recognition through a feed-forward neural network.

Chapter 3: System Analysis

3.1 Requirement Analysis

The requirement includes both functional and non- functional requirements:

3.1.1 Functional Requirements

Functional requirements define the specific tasks or operations that the system, or its components, must be capable of performing. These requirements can be effectively illustrated using Use Case Diagrams, which serve as a powerful tool for visualizing the system's functionality. Use Case Diagrams not only help translate functional requirements into design decisions and development priorities but also assist in identifying internal or external factors that may impact the system and should be considered during implementation. They provide a high-level overview of the system from an external perspective, focusing on how the system interacts with users (actors) without delving into the technical intricacies of how these functionalities are implemented. This approach ensures clarity in understanding the system's behavior and guides the development process effectively.

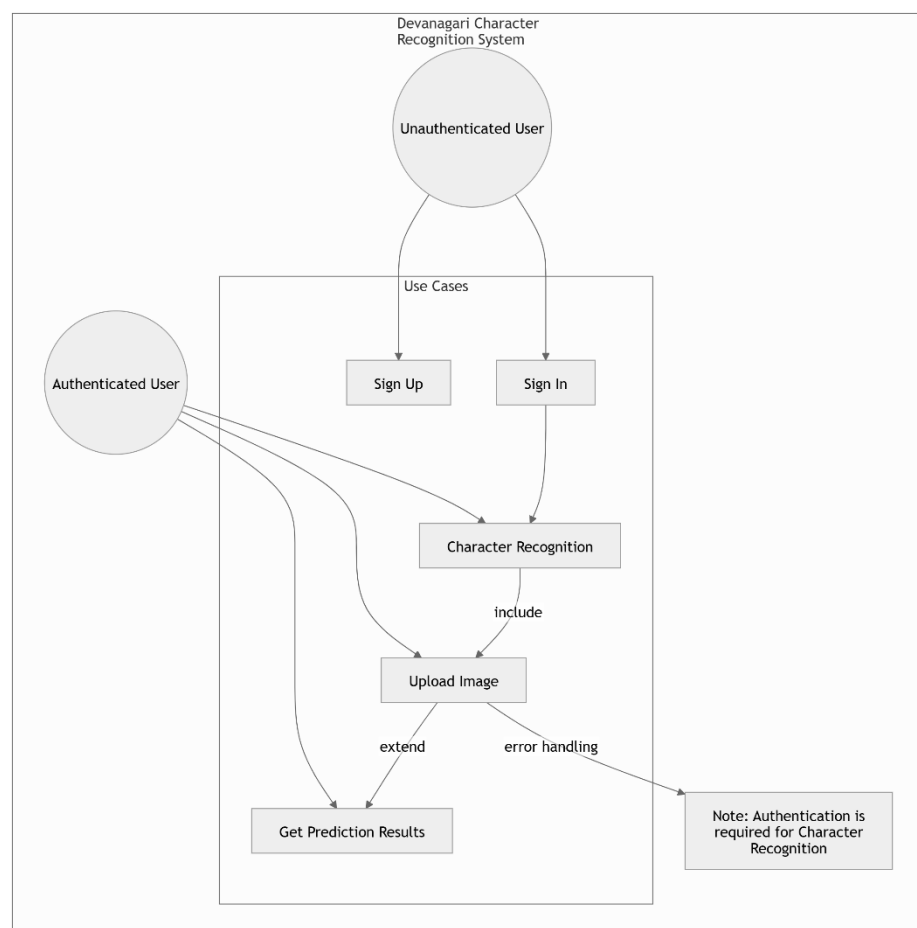


Figure 3. 1: Use Case Diagram

The Use Case Diagram of DCR consisted of the two actors i.e., user and computer. The actors interact with the DCR system shown in the middle rectangle. The use cases are represented in an oval shape. They include and extend links are used to show the relation between the use cases.

The Devanagari Handwritten characters should be drawn by the user. The character drawn is given to the DCR system directly using the file name where the image is saved or using the camera. The DCR system should have the functionality to scan the image given by the user. The scanning process includes pre-processing and segmentation functions serially. The DCR system should also consist the functionality to check the pixels using neural network for the purpose of recognizing the characters. The system then finally has the functionality to display the character recognized using the user interface.

The above-described functional requirements are mandatory for developing a well-functioning DCR system.

3.1.2 Non-Functional Requirements

The non-functional requirements are:

Performance

The DCR system must deliver accurate character recognition while maintaining efficient resource utilization. This includes optimizing memory, CPU, and storage usage to ensure smooth operation even under varying workloads. The system should minimize latency during character recognition and processing to provide a seamless user experience.

Scalability

The system must be designed to handle an increasing volume of characters as demand grows. Whether processing a single character or multiple characters simultaneously, the DCR system should scale efficiently without compromising performance or accuracy. This ensures the system remains robust and adaptable to future needs.

Reliability

Reliability is critical for the DCR system, as it must consistently classify individual characters with high accuracy. To achieve this, the system employs Convolutional Neural Networks (CNNs), which are specifically trained to recognize Devanagari characters. The use of CNNs ensures reliable predictions and minimizes errors in character classification.

Recoverability

In the event of incorrect predictions, the system should provide users with actionable feedback or alternative solutions. For example, it could suggest possible corrections or allow users to re-upload images for improved results. This recoverability feature enhances user trust and improves the overall usability of the system.

Usability

The DCR system is designed to be accessible to users familiar with Python and machine learning environments. It provides a user-friendly interface for uploading images and viewing recognition

results, while also offering flexibility for advanced users to customize or extend the system. Clear documentation and intuitive workflows further enhance usability.

Interoperability

The system integrates various machine learning frameworks and libraries within a Python-based environment. These components must operate seamlessly together to ensure compatibility and consistent performance. By adhering to industry standards and leveraging widely used tools, the DCR system achieves high interoperability, making it easier to maintain and extend.

3.2 Feasibility Analysis

The feasibility analysis assesses the viability and quality of a project, providing essential information to aid decision-making. A comprehensive feasibility study has been conducted to determine the potential success of the Devanagari Character Recognition (DCR) system. The following aspects of the feasibility analysis have been considered in evaluating the project's feasibility:

3.2.1 Economic Feasibility

The project is economically viable as it works on free, open-source tools like Python, Anaconda, and TensorFlow. Model training was performed on a local machine with 8GB RAM, which, while limited, proved sufficient for development. This eliminates the need for expensive hardware or cloud computing, making the job cost-effective.

3.2.2 Operational Feasibility

The Devanagari Character Recognition (DCR) system is designed to be operationally practical and user-friendly. It leverages a **Convolutional Neural Network (CNN)** to recognize Devanagari characters with a high degree of accuracy, while also accounting for a manageable error margin. The system is implemented as a Python-based program with a simple and intuitive user interface, ensuring ease of use for general users.

The demand for such a system is evident, particularly in regions like Nepal, where there is limited research and development in the field of **Natural Language Processing (NLP)** for Devanagari script. By addressing this gap, the DCR system fulfills a clear need, enhancing its operational relevance and usability. Users only need to install Python and the necessary dependencies to run the system, making it accessible even to those with basic technical knowledge.

3.2.3 Technical Feasibility

The technical feasibility of the Devanagari Character Recognition (DCR) system is ensured through the use of robust and widely-adopted technologies. The system is developed using Python, along with machine learning frameworks such as Keras and TensorFlow, which are well-suited for implementing CNN-based models.

The development team possesses strong technical expertise in machine learning algorithms, image processing, and neural network design, enabling the successful implementation of the system. Model training was conducted on a local machine with an Intel Core i5 10th Generation processor and, demonstrating that the system can be developed and tested effectively without requiring specialized hardware. This confirms the technical feasibility of

the project and ensures that the system can be deployed and maintained with existing resources.

3.2.4 Schedule Feasibility

The Gantt chart for the Devanagari Character Recognition (DCR) project is as follows:

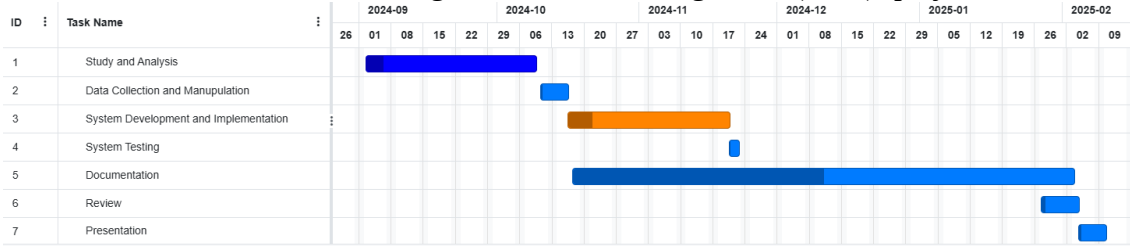


Figure 3. 1: Gantt Chart

The DCR system schedule consists of seven major activities. The Study and Analysis phase is planned to be completed within the first month. Data Collection and Manipulation is scheduled for the following 10 days. The Development and Implementation phase will begin in the third phase and is planned to run for one month. System Testing is scheduled to take place over the next 7 days. Documentation will begin after system development starts and is planned to be completed within 50 days. The System Review phase is scheduled for 10 days in the sixth phase. Finally, the Presentation will be conducted in the last phase, lasting for 5 days.

3.3 Constructing System Requirements

• Process Modeling

Process modeling visually depicts the processes that acquire, process, store, and disseminate data between a system and its environment, as well as among the system's components. Every process converts inputs into outputs. The Data Flow Diagram (DFD) is commonly used for process modeling. A DFD is a pictorial representation of the movement of data between entities, processes, and data stores within a system. DFDs can be used to analyze an existing system or model a new one. Entities are represented by rectangular boxes, processes are depicted as circles, and directional links indicate the flow of data between entities and processes.

The DFD of the Devanagari Character Recognition (DCR) system is shown on the next page. The diagram includes three levels of DFD, with each level providing a more detailed analysis of the data flow. As we proceed to deeper levels of analysis, we move to the next level of the DFD. The three levels are described below, with the higher level offering more detail than the consecutive lower levels.

1. Level 0 DFD:

The Level 0 DFD is the simplest representation of the data flow within the system, providing a basic overview of the flow of data. It shows that the input image is fed into the Devanagari Character Recognition (DCR) system, and the system outputs the recognized character.

2. Level 1 DFD:

The Level 1 DFD is an expansion of the Level 0 DFD. In this level, the input image is provided to the Devanagari Character Recognition (DCR) system through the user interface. The scanned image is then pre-processed, which involves finding the borders and segmenting the image if it contains multiple characters. The feature extraction process extracts relevant features from the pre-processed character image. These extracted features are then passed to the character recognition process, and the recognized character is displayed to the user.

3. Level 2 DFD:

Level 2 DFD is the more detailed expansion of process in level 1 DFD. The Level 2 DFD shown in the figure is the division of the process 1.4 in level 1 DFD into two sub processes i.e. 1.4.1 and 1.4.2. At first the co-relation is calculated for all Nepali alphabets using the extracted features. Then the maximum co-related value is obtained. The alphabet with the maximum co-related value is displayed as the recognized character.

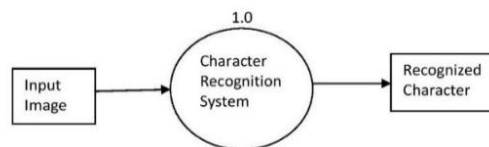


Fig 1: Level 0 DFD

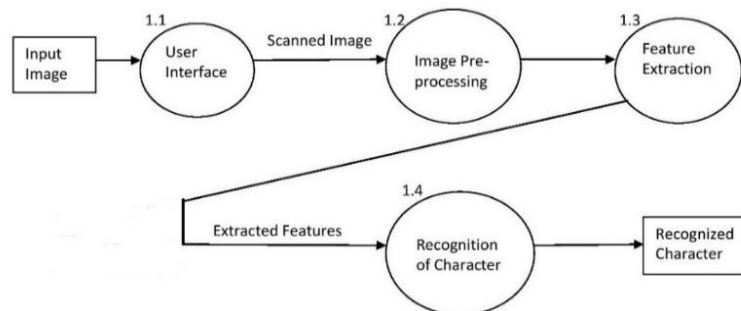


Fig 2: Level 1 DFD

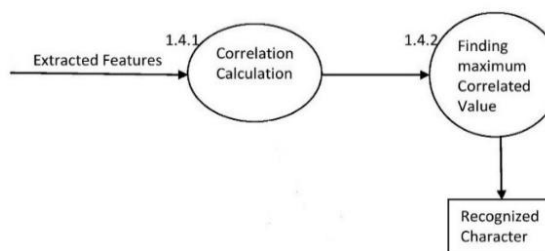


Fig 3: Level 2 DFD

Figure 3. 2: DFD

Chapter 4: System Design and Implementation

4.1 System Design

4.1.1 Block Diagram

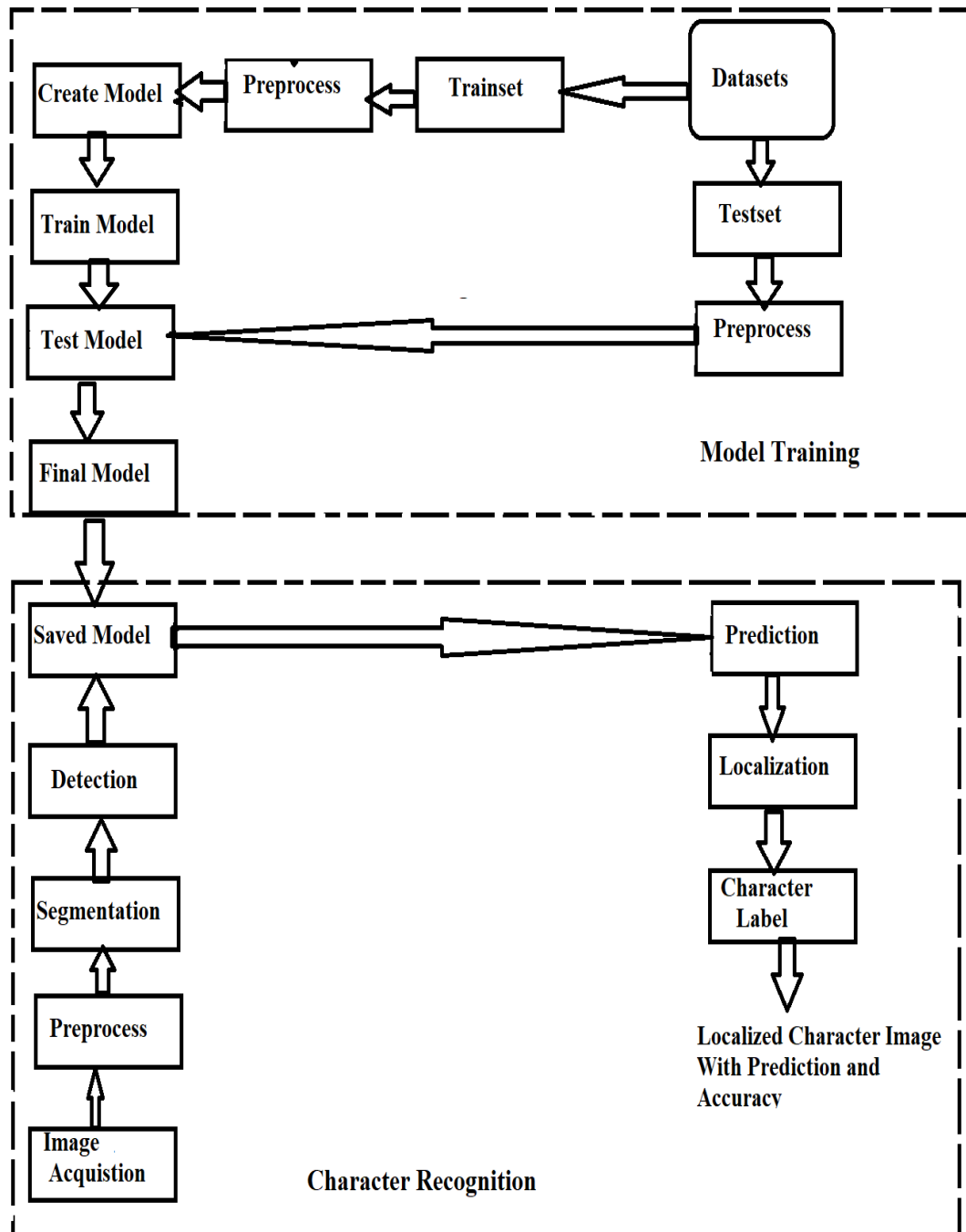


Figure 4. 1:Block Diagram of DCR system

Model Training:

The training set is created from a dataset downloaded from an online source. Following this, the training dataset undergoes pre-processing. A neural network model is then constructed using the Keras framework. The pre-processed training set is passed into the created neural network model, which is trained accordingly to produce a test model.

Similarly, the test set is developed from the dataset, pre-processed, and passed to the previously created test model. Once the desired accuracy is achieved, the test model is finalized, and the final model is saved.

Character Recognition:

In the first step, the raw image is acquired using local storage. This image is then pre-processed to extract relevant features. If the image contains more than one character, it is segmented. The segmented image is passed to the saved model, which predicts the characters, localizes the image, and displays the recognized characters to the user.

4.1.2 Dataset Preparation

The dataset used in the project is publicly available on the web and is divided into two parts: training and testing. After downloading the dataset, it is converted into the appropriate format to feed into the neural network (NN). The dataset consists of grayscale images, with different collections of training and test images. The training set contains 78,200 examples, with 1,700 examples for each character, while the test set has 200 examples for each character.

In the images, the foreground is white, and the background is black. Each image contains a 2-pixel border around the character. The character itself is a 28x28 pixel grayscale image, and with the 2-pixel border on each side, the final image size becomes 32x32 pixels. The images are in PNG format, and each example is stored within a folder labeled according to the character.

To train the model efficiently, instead of moving the entire image file back and forth between folders, which increases runtime complexity, all images are saved into a CSV file. Each image is converted into pixel values ranging from 0 to 255. The first column of the CSV file represents the label, and the remaining 1,024 columns contain the pixel values. Labeling is done serially from 0 to 46 (e.g., 0 represents '0', 10 represents 'ka', and 46 represents 'gya'). Both training and testing images are saved into individual CSV files. This process takes approximately 10 minutes to complete on a local device. However, the size of the CSV file is larger than the combined size of the original image files.

4.2 Implementation

4.2.1 Implementation in Python

All coding for this project concluded using Python 3.10, leveraging a robust set of libraries to ensure efficient development and execution. The implementation relies heavily on NumPy for high-performance array operations, which are important for processing image data and handling mathematical computations. OpenCV is utilized widely for advanced image processing tasks, including pre-processing, noise removal, and character segmentation. For visualizations and debugging purposes, Matplotlib is employed to create graphs and analyse results effectively. The neural network model is built using Keras, a powerful deep learning system that uses TensorFlow as its backend. This combination allows seamless GPU-based training, significantly reducing the time needed for model optimization and prediction.

Additionally, the system uses gTTS (Google Text-to-Speech) for generating audio outputs in the Nepali language, enhancing the overall usefulness of the application. These tools, combined with Python's built-in libraries, form the basis of the Devanagari Character Recognition System, ensuring accurate character recognition and efficient processing.

4.3 Neural Network Design and Implementation

4.3.1 Convolutional Neural Network

The **Devanagari Character Recognition System** employs a Convolutional Neural Network (CNN) for accurate recognition of Devanagari characters. The architecture is designed to minimize the number of learning parameters while maintaining high accuracy. Below is a detailed description of the architecture and its implementation:

Architecture Parameters

Each layer in the CNN has been carefully configured to optimize performance and accuracy. The key parameters are as follows:

- **Number of Filters:** 32 (first convolutional layer), 64 (second convolutional layer)
- **Filter Size:** (5, 5) for all convolutional layers
- **Pooling Size:** (2, 2) for the first pooling layer, (5, 5) for the second pooling layer
- **Activation Function:** ReLU for all layers except the final layer, which uses **Softmax** for multi-class classification
- **Dropout:** Not explicitly used in this architecture but can be added to prevent overfitting if needed

The architecture is as follows:

Layer Name	Type	Output Shape	Parameters
conv2d	Conv2D	(None, 28, 28, 32)	832
max_pooling2d	MaxPooling2D	(None, 14, 14, 32)	0
conv2d_1	Conv2D	(None, 10, 10, 64)	51,264
max_pooling2d_1	MaxPooling2D	(None, 2, 2, 64)	0
flatten	Flatten	(None, 256)	0
dense	Dense	(None, 37)	9,509
Total Parameters			184,817
Trainable Parameters			61,605
Non-Trainable Parameters			0
Optimizer Parameters			123,212
CNN Error			3.95%

Figure 4. 2: CNN Architecture

Training Configuration

The model was trained with the following configuration:

- Loss Function: Categorical Crossentropy
- Optimizer: Adam
- Batch Size: 64
- Epochs: 8
- Validation Split: Implicitly handled by splitting the dataset into training (70,000 samples) and testing (2,000 samples) sets
- Training Time: Approximately 40 minutes on Local PC
- Test Accuracy: 96.05%
- Error Rate: 3.95%

4.4 Image Processing Model

The training of the Convolutional Neural Network (CNN) model has been successfully completed, and the system is now capable of performing real-world Devanagari character recognition. The primary objective of this system is to convert images containing Devanagari characters—whether typed, handwritten, or printed—into machine-encoded text using advanced image processing techniques and a robust neural network. In this implementation, users upload images through the web application. These images are either stored locally or temporarily processed on the server for prediction. Once uploaded, the system performs a series of preprocessing steps, including noise removal, segmentation, and normalization, before passing the processed images to the CNN model for accurate prediction via REST API endpoints.

4.4.1 Image Acquisition

The image acquisition stage is a critical component of the recognition process. Even with a highly accurate model, real-world images can vary significantly due to factors such as noise, blur, and other quality degradation issues. The method used for capturing images also impacts their quality.

In our approach, we have utilized locally stored images for processing. These images are pre-saved in a designated directory and retrieved for recognition. This ensures consistency in image quality and reduces variations caused by external acquisition devices such as cameras or scanners. By working with locally stored images, we maintain control over image resolution, format, and preprocessing, leading to more stable recognition results.

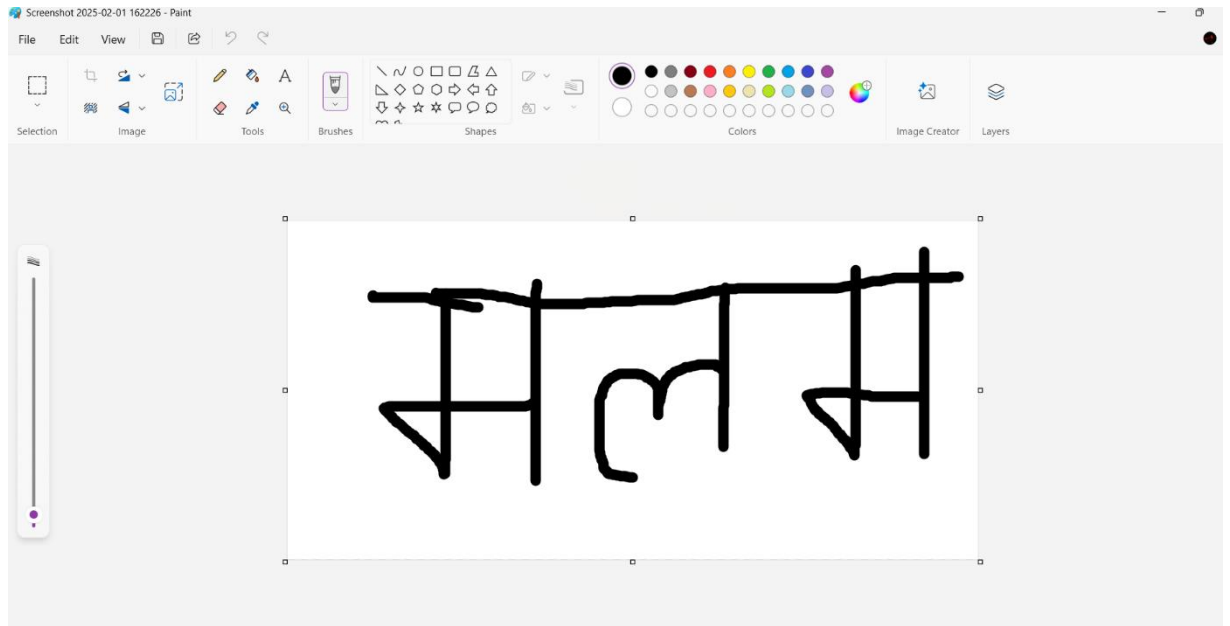


Figure 4. 3: Character Creation using paint

4.4.2 Image Preprocessing

Image preprocessing is a crucial step in the recognition pipeline, as it enhances image quality and prepares it for character extraction. The primary goal at this stage is to separate the foreground (text) from the background, reduce noise, and standardize the input format for prediction. The process begins by converting the image to grayscale, ensuring that pixel intensity values are uniform and independent of color variations. To further refine the image, thresholding techniques are applied to create a binary representation. This method effectively differentiates the text from the background, making character recognition more reliable.

For thresholding, a combination of **Otsu's method** and **binary thresholding** is employed. Otsu's technique automatically determines an optimal threshold value by analyzing the pixel intensity distribution. It assumes a bi-modal histogram, where the pixels are classified into two distinct groups—foreground and background. The algorithm selects the threshold that minimizes intra-class variance, ensuring a clear distinction between text and background.

Despite its effectiveness, thresholding can sometimes misclassify foreground and background pixels, particularly when dealing with images affected by uneven lighting. High brightness levels or reflections can introduce inconsistencies, leading to incorrect character detection. To mitigate this, clear and bold text with minimal background noise is recommended for accurate recognition. To further ensure precise detection, a technique is implemented to verify the foreground and background colors. This involves sampling a few pixels along the diagonal of the image, allowing the system to identify whether the text is dark or light against its background.

4.4.3 Segmentation

Segmentation is the process of isolating individual characters from the input image. This step is critical when an image contains multiple characters that need to be processed separately. In our approach, segmentation is performed by analyzing pixel distribution and detecting spaces between characters. The function scans the image from left to right, identifying distinct character boundaries. Once a separation is detected, the segment is extracted and prepared for prediction.

However, segmentation is challenging for characters like ण and ण, which may contain gaps within their structure. To prevent misinterpretation, an adaptive spacing criterion is used. This ensures that the segmentation function differentiates between actual character separations and internal gaps within individual letters.

Each segmented portion is stored in an array and passed to the prediction function. This method ensures that the system processes only meaningful character segments, reducing errors in recognition.

4.4.4 Character Prediction/classification

Once segmentation is complete, each character segment is resized to match the input dimensions required by the trained neural network. In this case, each segment is resized to 30x30 pixels, with a 1-pixel border added, making the final size 32x32 pixels. The processed segment is then passed to the CNN model, which predicts the character by analyzing its features. If the prediction confidence is high, the character is recognized and stored as output. However, if confidence is low, additional refinement techniques are applied to enhance recognition accuracy.

A potential issue encountered during this stage is related to image resizing. If the total number of pixels exceeds a specific threshold, OpenCV's `size.area()` function may return an invalid value, leading to resizing failures. To prevent this, an assertion check is implemented, ensuring that images are processed correctly before being passed to the model.

Additionally, while the model is trained specifically on Devanagari characters, it may occasionally predict incorrect results when given irrelevant images. Currently, the system does not include a mechanism to reject non-Devanagari inputs, meaning that users must ensure that only valid characters are provided for recognition.

4.4.5 Character Localization

Character localization is an enhancement of segmentation that focuses on identifying the exact position of characters within the original image. This step helps refine the segmentation process by reducing false detections and ensuring accurate predictions. Using **template matching**, each character segment is compared against the original image, and bounding boxes are drawn around detected regions. This method eliminates unnecessary spaces between characters and enhances the accuracy of recognition.

By refining character positions and ensuring proper alignment, localization improves overall prediction quality and reduces errors caused by incorrect segmentation.

4.4.6 Character Recognition

The character recognition phase is the culmination of all preceding steps, combining image pre-processing, segmentation, and classification to accurately identify Devanagari characters. At this stage, the system processes each segmented character, applies the trained convolutional neural network (CNN) model, and assigns the most probable label based on the model's predictions. The model analyzes distinct character features such as curves, edges, and stroke patterns to make accurate classifications. Bounding boxes are drawn around detected characters, highlighting the recognized text within the processed image. This step ensures that the output is structured and visually interpretable.

The recognition process depends heavily on the quality of the input image. If the image contains noise, distortion, or poor lighting conditions, the accuracy of predictions may decrease, leading to false identifications. To mitigate such issues, high-resolution images with clear text and minimal background interference are recommended. Additionally, the model's accuracy can be improved through continuous training with diverse datasets, enabling it to handle various handwriting styles and challenging scenarios. Despite minor limitations, this recognition system offers reliable performance in extracting Devanagari characters, making it well-suited for applications in digitization, text conversion, and automated transcription.

4.5 List of Algorithms

4.5.1 Load Dataset

- Step 1:** Start
- Step 2:** Read the dataset file from local storage.
- Step 3:** Shuffle the dataset randomly to avoid bias in training.
- Step 4:** Separate the dataset into features (x) and labels (y).
- Step 5:** Normalize pixel values by dividing by 255.
- Step 6:** Convert labels into categorical format.
- Step 7:** Reshape the data into a format compatible with the CNN model.
- Step 8:** Stop.

4.5.2 Create CNN Model

Steps for CNN Model

- Step 1:** Start.
- Step 2:** Define input shape as (32, 32, 1).
- Step 3:** Add the first convolutional layer with 32 filters, kernel size (5,5), activation ReLU, and apply max pooling with a 2x2 filter and same padding.
- Step 4:** Add the second convolutional layer with 64 filters, kernel size (5,5), activation ReLU and apply max pooling with a 5x5 filter and the same padding.
- Step 5:** Flatten the feature maps.
- Step 6:** Add a dense layer with 37 output neurons and softmax activation.
- Step 7:** Compile the model using categorical cross-entropy loss and Adam optimizer.
- Step 8:** Stop.

4.5.3 Train CNN Model

- Step 1:** Start
- Step 2:** Define batch size, epochs, and training parameters.
- Step 3:** Begin training using X_{train} and Y_{train} .
- Step 4:** For each epoch:
 - Compute loss and accuracy on training data.
 - Validate performance on test data.
 - If validation accuracy improves, save the model checkpoint.
- Step 5:** Stop training when the desired accuracy is reached.
- Step 6:** Stop.

4.5.4 Image Processing

A. Image Preprocessing

- Step 1:** Start
- Step 2:** Convert input image to grayscale.
- Step 3:** Use Contrast Limited Adaptive Histogram Equalization (CLAHE) to enhance contrast.
- Step 4:** Apply Otsu's thresholding to obtain a binary image.
- Step 5:** Remove noise using morphological operations (opening and closing).
- Step 6:** Identify foreground and background colors using diagonal pixel sampling.
- Step 7:** Normalize pixel values and reshape to (32, 32, 1).
- Step 8:** Stop.

B. Image Segmentation

- Step 1:** Start
- Step 2:** Detect connected components in the binary image.
- Step 3:** Identify character regions based on bounding boxes.
- Step 4:** Group connected components into character clusters.
- Step 5:** Crop each character region to extract individual segments.
- Step 6:** Resize segments to 32×32 for input into the CNN.
- Step 7:** Stop.

4.5.5 Character Recognition & Prediction

- Step 1:** Start
- Step 2:** Load the trained CNN model.
- Step 3:** For each segmented character:
 - Convert it into a 32×32 grayscale image.
 - Normalize pixel values.
 - Pass it through the CNN model for prediction.
- Step 4:** Obtain predicted character and confidence score.
- Step 5:** If confidence is above the threshold, store the predicted character.
- Step 6:** Combine predicted characters to form the recognized text.
- Step 7:** Display recognized text and overlay bounding boxes on the image.
- Step 8:** Stop.

4.5 Usage of Algorithms in Our Project

The algorithms used in this project form a structured pipeline for **Devanagari character recognition**. The **dataset loading algorithm** ensures proper data preparation by normalizing and reshaping images for efficient model training. The **CNN model creation algorithm** constructs a deep learning model with convolutional layers to extract meaningful features from handwritten characters. The **training algorithm** optimizes the model using labeled data, improving accuracy through multiple epochs and validation checks.

The **image processing algorithm** refines the input by converting images to grayscale, removing noise, and applying thresholding to differentiate text from the background. The **segmentation algorithm** isolates individual characters by detecting connected components and ensuring proper separation. Finally, the **recognition algorithm** predicts characters from segmented images using the trained CNN model, reconstructing the text with high accuracy. These algorithms collectively enable robust character recognition, making the system efficient for real-world applications.

Chapter 5: System Testing

The testing phase can be conducted manually or using automated tools to validate the functionality of each component. After development, all system components undergo rigorous testing to ensure quality, performance, and the absence of technical issues. Testing also helps identify discrepancies between expected and actual functionality. As a critical step in the development cycle, it ensures that every part of the system operates as intended.

5.1 Unit Testing

During the coding phase, each module underwent rigorous testing to verify its functionality. Any errors detected during unit testing were promptly debugged and resolved. Below are some of the key test cases that were executed:

Test Case 1: API Authentication Flow Testing

Inputs:

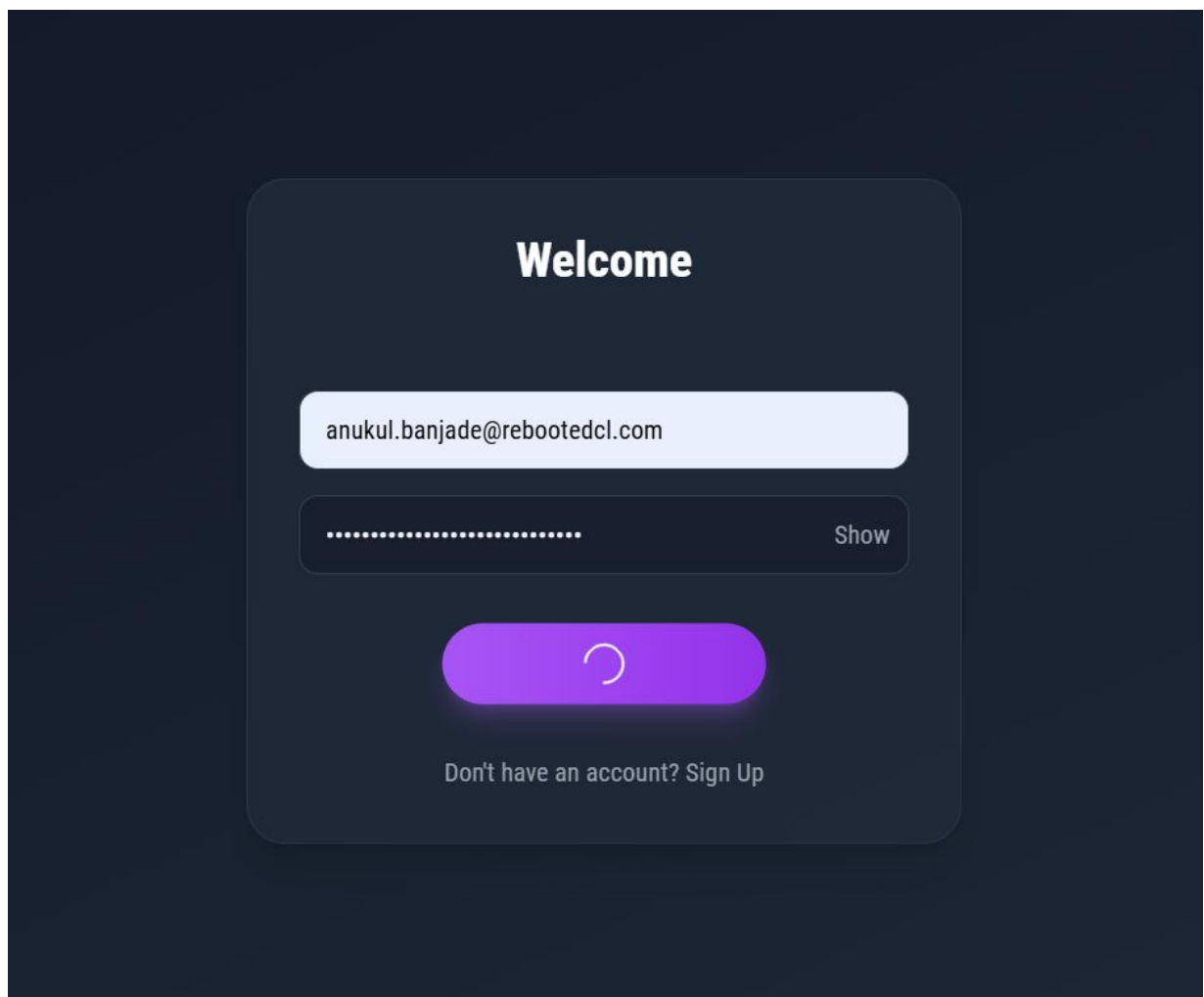


Figure 5.1.1: Login screen

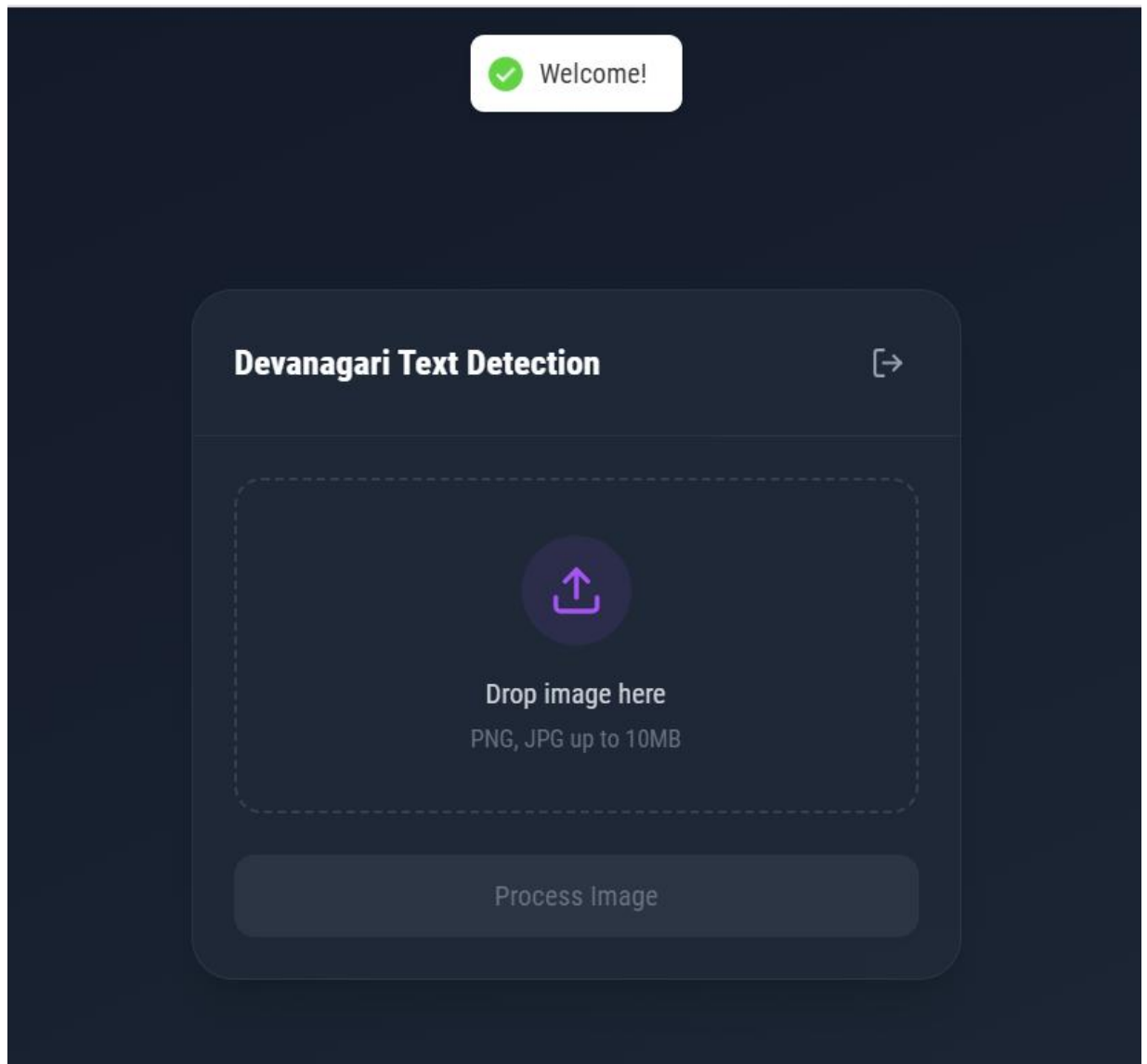


Figure 5.1.2: Login Success

Input: User signup/signin credentials

Process: Testing token generation and validation

Results:

- Successful token generation for valid credentials
- Proper error handling for invalid credentials
- Correct implementation of JWT expiration

Test Case 2: Model Prediction Integration

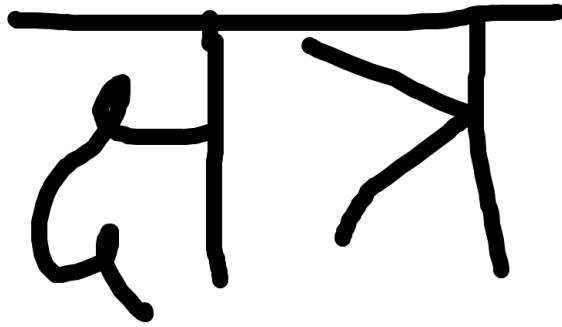
Testing Process:

Input: Segmented character images

Process: Testing integration with Keras model

Results: Recognized Successfully

Input Image:



Results:

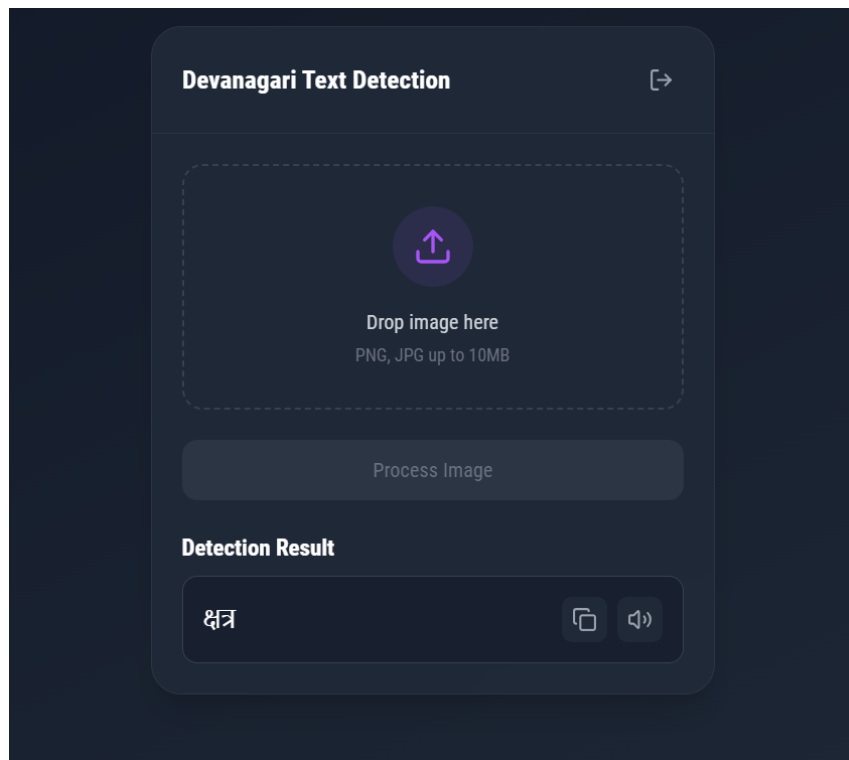


Fig 5.1.3: Model Prediction Integration

5.2 Integration Testing

Integration testing combines individually tested components and validates their combined functionality. This testing phase verifies the seamless interaction between different modules while identifying and resolving interface-related issues.

Key integration test cases include:

Test Case 1: Integration Testing of File Upload and Preprocessing Module

In this test, the file upload functionality and image preprocessing components are integrated to ensure proper handling of input images. The system validates image formats, performs size checks, and prepares images for character recognition.

Test Case 1: Results:

- Image format validation successfully filters PNG/JPG files
- File size restrictions (16MB limit) function correctly
- Binary to grayscale conversion produces consistent results

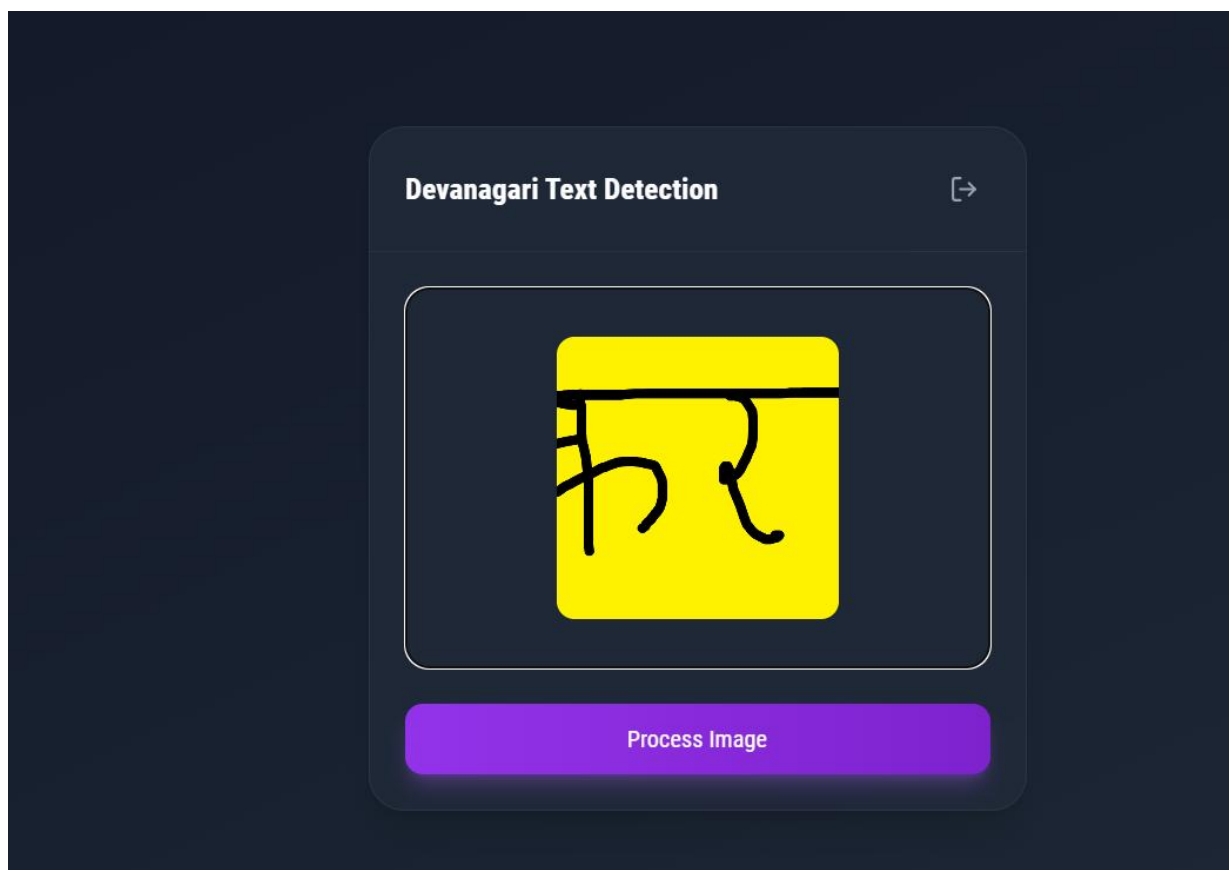


Fig 5.2.1: Integration Testing of File Upload

Test Case 2: Integration of Preprocessing with Character Segmentation

After confirming successful image preprocessing, the character segmentation module integration is tested. This phase examines the system's ability to identify and isolate individual Devanagari characters.

Test Case 2: Results:

- Connected component analysis accurately identifies character regions
- Header line (“dikka”) removal works effectively
- Character grouping logic handles varying character widths
- Proper padding maintains character proportions

Test Case 3: Integration Testing of Complete Recognition Pipeline

The final integration phase combines preprocessing, segmentation, and the neural network model for character recognition. This test validates the end-to-end character recognition process.

Test Case 3: Results:

- Model loading and initialization functions reliably
- Prediction confidence scores reflect accuracy levels
- Character mapping produces correct Devanagari output

Test Case 4: Authentication and User Management Integration

This phase tests the integration of user authentication with core functionality.

Test Case 5: Results:

- JWT token generation and validation works consistently
- Protected routes properly verify authentication
- User profiles maintain prediction

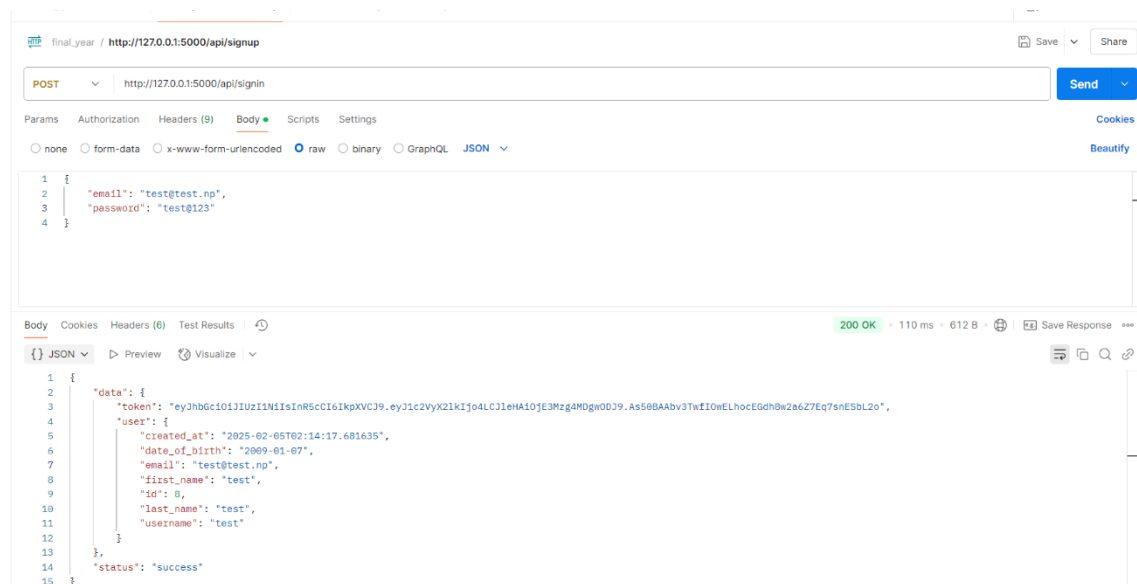


Fig 5.2.2: API Testing using Postman

5.3 System Testing

System testing was performed following integration testing to validate the complete application functionality. This phase evaluated the entire workflow from user authentication through character recognition to audio generation, confirming alignment with system requirements. Through thorough testing, several critical performance considerations were identified.

Results:

Server Performance:

- Extended server uptime impacts response times, particularly during high traffic periods
- Database query optimization becomes crucial when handling multiple prediction requests
- Memory management requires attention when processing multiple image uploads simultaneously

Image Processing Efficiency:

- Image preprocessing time varies significantly based on file size and resolution
- Connected component analysis performance scales with character density
- Character segmentation processing time increases with complex character arrangements
- Higher resolution uploads require additional computational resources

API Response Times:

- Authentication token validation adds consistent overhead to each request
- Concurrent user sessions impact overall system responsiveness
- File upload bandwidth affects initial processing delay
- Audio generation time scales with text length

Performance Optimization Areas:

- Image compression could improve upload and processing speeds
- Caching frequently accessed user data reduces database load
- Implementing request queuing helps manage peak traffic periods
- Memory clean up between predictions maintains consistent performance

The system maintains accuracy across various conditions but requires performance monitoring during extended operation periods. Response times remain dependent on input complexity and server resource availability.

Chapter 6: Maintenance and Support

6.1 Maintenance

The Devanagari Character Recognition application operates within a carefully structured technical environment, combining modern web technologies with machine learning capabilities. The system utilizes a React 18.3.1 frontend built with Vite, communicating with a Flask backend that handles image processing and character recognition tasks. This architecture supports essential features including user authentication via JWT tokens, image upload processing with size limitations of 16MB, and audio generation for recognized text.

Performance characteristics reveal specific timing patterns across different operations, with image preprocessing taking approximately 0.5 seconds, character segmentation requiring 1.5-2 seconds, and model prediction consuming 1-1.5 seconds. The complete processing pipeline, including audio generation, typically completes within 3.5-4 seconds. This timing breakdown highlights areas where resource utilization peaks, particularly during character segmentation and model inference phases. The system's response time can vary based on factors such as concurrent user requests, image complexity, and server resource availability.

Several optimization opportunities exist within the current implementation. Image processing could benefit from enhanced preprocessing algorithms and more efficient connected component analysis. Resource management improvements could include implementing strategic memory cleanup routines and optimizing database query patterns. Additionally, implementing response caching mechanisms and fine-tuning preprocessing parameters could lead to improved overall system performance. These enhancements would need to balance processing efficiency with recognition accuracy to maintain reliable character detection while reducing resource consumption.

6.2 Support

The Devanagari Character Recognition project implements an efficient development cycle using modern web technologies. The system combines React 18.3.1 frontend development with a Flask backend, enabling streamlined testing and validation procedures. This architecture allows for quick adjustments based on performance feedback and testing results.

The implementation balances technical capabilities with resource constraints, focusing on sustainable optimization strategies. Key areas for enhancement include preprocessing algorithm efficiency, database query optimization, and memory management improvements. While hardware upgrades could boost processing speed, the current architecture emphasizes efficient resource utilization through strategic caching and streamlined workflows, ensuring consistent performance within existing system limitations.

Chapter 7: Conclusion & Future enhancements

7.1 Conclusion

The Devanagari Character Recognition project demonstrates the practical application of machine learning and web technologies in addressing character recognition challenges. By combining React-based frontend development with Flask backend processing, the system successfully implements character recognition functionality with audio output generation. The implementation showcases effective integration of multiple technologies including JWT authentication, image processing with OpenCV, and neural network-based character prediction. While the current version operates within specific hardware and software constraints, achieving recognition times of 3.5-4 seconds per image, it establishes a foundation for future improvements.

This development process enhanced our understanding of full-stack application architecture and machine learning integration. Working through challenges in image processing, database management, and user authentication provided valuable insights into building complex systems. The project highlighted the importance of balancing processing efficiency with recognition accuracy while maintaining sustainable resource utilization. These experiences contributed significantly to our technical expertise and project management capabilities.

7.2 Future enhancements

Potential areas for system expansion include:

1. Development of responsive mobile applications for iOS and Android platforms
2. Implementation of cross-language text conversion capabilities
3. Enhancement of complex character recognition algorithms
4. Integration with cloud services for improved scalability
5. Addition of advanced caching mechanisms for faster processing
6. Implementation of real-time character recognition features
7. Integration of natural language processing capabilities.

Bibliography

- Chhabra, A. (2019). *Deep learning based real-time Devanagari character recognition*. San Jose: SJSU ScholarWorks. <https://doi.org/10.31979/etd.3yh5-xs5s>
- Deshmukh, A., Meshram, R., Kendre, S., & Shah, K. (2014). *Handwritten Devanagari character recognition*. *International Journal of Engineering Research*, Pune.
- Dongare, S. A., Kshirsagar, D. B., & Waghchaure, S. V. (2014). *Handwritten Devanagari character recognition using neural network*. *IOSR Journal of Computer Engineering (IOSR-JCE)*, Kopargaon.
- Gyawali, K. P., Acharya, S., & Pant, A. (2016). *Deep learning-based large-scale handwritten Devanagari character recognition*. *Computer Vision Research Group*, Kathmandu.
- Narang, V., Roy, S., Murthy, O. V., & Hanmandlu, M. (2013). *Devanagari character recognition in scene images*. *IEEE*, Washington, DC.
- Negi, V., Mann, S., & Chauhan, V. (2017). *Devanagari character recognition using artificial neural network*. *International Journal of Engineering and Technology (IJET)*, New Delhi. <https://doi.org/10.21817/ijet/2017/v9i3/1709030246>
- Sayyad, S. S., Jadhav, A., Jadhav, M., Miraje, S., Bele, P., & Pandhare, A. (2013). *Devanagari character recognition using neural networks*. *International Journal of Engineering and Innovative Technology (IJEIT)*, Ashta.
- Vats, I., & Singh, S. (2014). *Offline handwritten English numerals recognition using correlation method*. *IJERT*, Chandigarh.

APPENDIX

Appendix A: Source Code

i. Model Training

```
import numpy as np
import pandas as pd
from keras import layers

from keras.layers import Input, Dense, Activation, ZeroPadding2D, BatchNormalization,
Flatten, Conv2D

from keras.layers import AveragePooling2D, MaxPooling2D, Dropout,
GlobalMaxPooling2D, GlobalAveragePooling2D

from keras.utils import to_categorical
from keras.models import Sequential
from keras.callbacks import ModelCheckpoint


data = pd.read_csv("data.csv")
dataset = np.array(data)
np.random.shuffle(dataset)
X = dataset
Y = dataset
X = X[:, 0:1024]
Y = Y[:, 1024]


X_train = X[0:70000, :]
X_train = X_train / 255.
X_test = X[70000:72001, :]
X_test = X_test / 255.


Y = Y.reshape(Y.shape[0], 1)
Y_train = Y[0:70000, :]
Y_train = Y_train.T
Y_test = Y[70000:72001, :]
Y_test = Y_test.T


image_x = 32
image_y = 32
```

```

train_y = to_categorical(Y_train)
test_y = to_categorical(Y_test)
train_y = train_y.reshape(train_y.shape[1], train_y.shape[2])
test_y = test_y.reshape(test_y.shape[1], test_y.shape[2])
X_train = X_train.reshape(X_train.shape[0], image_x, image_y, 1)
X_test = X_test.reshape(X_test.shape[0], image_x, image_y, 1)

def keras_model(image_x, image_y):
    num_of_classes = 37
    model = Sequential()
    model.add(Conv2D(filters=32, kernel_size=(5, 5), input_shape=(image_x, image_y, 1),
activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2), padding='same'))
    model.add(Conv2D(64, (5, 5), activation='relu'))
    model.add(MaxPooling2D(pool_size=(5, 5), strides=(5, 5), padding='same'))
    model.add(Flatten())
    model.add(Dense(num_of_classes, activation='softmax'))
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    filepath = "devanagari.h5"
    checkpoint1 = ModelCheckpoint(filepath, monitor='val_acc', verbose=1,
save_best_only=True, mode='max')
    callbacks_list = [checkpoint1]
    return model, callbacks_list

model, callbacks_list = keras_model(image_x, image_y)
model.fit(X_train, train_y, validation_data=(X_test, test_y), epochs=8, batch_size=64,
callbacks=callbacks_list)
scores = model.evaluate(X_test, test_y, verbose=0)
print("CNN Error: %.2f%%" % (100 - scores[1] * 100))
model.summary()
model.save('devanagari.h5')

```

ii. Image Processing

```
import cv2
```

```

import numpy as np
from keras.models import load_model

# Load the model
model = load_model('models/devanagari.h5')

# Character mapping
letter_map = {
    0: 'CHECK', 1: 'क', 2: 'ख', 3: 'ग', 4: 'घ', 5: 'ङ', 6: 'च',
    7: 'छ', 8: 'ज', 9: 'झ', 10: 'ञ', 11: 'ट', 12: 'ठ', 13: 'ड',
    14: 'ढ', 15: 'ण', 16: 'त', 17: 'थ', 18: 'द', 19: 'ध', 20: 'न',
    21: 'प', 22: 'फ', 23: 'ब', 24: 'भ', 25: 'म', 26: 'य', 27: 'र',
    28: 'ल', 29: 'व', 30: 'श', 31: 'ष', 32: 'स', 33: 'ह',
    34: 'क्ष', 35: 'त्र', 36: 'ज्ञ'
}

def remove_header_line(img):
    """Remove the shirorekha (header line) using improved method."""
    height, width = img.shape
    horiz_proj = np.sum(img, axis=1)
    upper_half = horiz_proj[:height//2]
    header_position = np.argmax(upper_half)
    header_thickness = height // 15
    mask = np.ones_like(img)
    mask[max(0, header_position-header_thickness):min(height,
header_position+header_thickness), :] = 0
    return img * mask

def get_connected_components(binary_img):
    """Get connected components with their bounding boxes and centroids."""
    num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(binary_img,
connectivity=8)
    stats = stats[1:]
    centroids = centroids[1:]

```



```

components = []
for i, (stat, centroid) in enumerate(zip(stats, centroids)):
    x, y, w, h, area = stat
    if area > 50:
        components.append({
            'label': i + 1,
            'x': x,
            'y': y,
            'w': w,
            'h': h,
            'area': area,
            'centroid': centroid
        })
return components

```

```

def merge_components(components, img_width):
    """Merge components into character groups based on spatial relationships."""
    if not components:
        return []
    components = sorted(components, key=lambda c: c['x'])
    total_width = components[-1]['x'] + components[-1]['w'] - components[0]['x']
    avg_char_width = total_width / 3
    char_groups = []
    current_group = [components[0]]
    for i in range(1, len(components)):
        curr_comp = components[i]
        prev_comp = current_group[-1]
        distance = curr_comp['x'] - (prev_comp['x'] + prev_comp['w'])
        should_merge = (
            distance < avg_char_width * 0.3 or
            curr_comp['x'] < prev_comp['x'] + prev_comp['w'] or
            len(char_groups) == 2
        )
        if should_merge:
            current_group.append(curr_comp)

```

```

        else:
            char_groups.append(current_group)
            current_group = [curr_comp]
        char_groups.append(current_group)
    if len(char_groups) != 3:
        char_width = img_width // 3
        new_groups = [[], [], []]
        for comp in components:
            group_idx = min(2, comp['x'] // char_width)
            new_groups[group_idx].append(comp)
        char_groups = new_groups
    return char_groups

def extract_character_from_group(group, binary_img):
    """Extract character image from a group of components."""
    if not group:
        return None
    min_x = min(comp['x'] for comp in group)
    min_y = min(comp['y'] for comp in group)
    max_x = max(comp['x'] + comp['w'] for comp in group)
    max_y = max(comp['y'] + comp['h'] for comp in group)
    char_img = binary_img[min_y:max_y, min_x:max_x]
    padding = 5
    char_img = cv2.copyMakeBorder(
        char_img,
        padding, padding, padding, padding,
        cv2.BORDER_CONSTANT,
        value=0
    )
    return char_img

def process_image(image_data):
    """Process image data for prediction."""
    nparr = np.frombuffer(image_data, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_GRAYSCALE)

```

```

target_height = 150
aspect_ratio = img.shape[1] / img.shape[0]
target_width = int(target_height * aspect_ratio)
img = cv2.resize(img, (target_width, target_height))
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
img = clahe.apply(img)
_, binary = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)
binary_no_header = remove_header_line(binary)
kernel = np.ones((2,2), np.uint8)
binary_clean = cv2.morphologyEx(binary_no_header, cv2.MORPH_OPEN, kernel)
binary_clean = cv2.morphologyEx(binary_clean, cv2.MORPH_CLOSE, kernel)
components = get_connected_components(binary_clean)
char_groups = merge_components(components, binary_clean.shape[1])
segments = []
for group in char_groups:
    if group:
        char_img = extract_character_from_group(group, binary_clean)
        if char_img is not None:
            segments.append(char_img)
return segments

def process_segment(segment):
    """Process individual segment for prediction."""
    _, segment = cv2.threshold(segment, 127, 255, cv2.THRESH_BINARY)
    processed = cv2.resize(segment, (32, 32))
    processed = np.array(processed, dtype=np.float32) / 255.0
    processed = np.reshape(processed, (1, 32, 32, 1))
    return processed

def predict_segments(segments):
    """Predict characters from segments."""
    predictions = []
    confidences = []
    for segment in segments:

```

```

processed = process_segment(segment)
pred_probab = model.predict(processed)[0]
pred_class = np.argmax(pred_probab)
confidence = float(pred_probab[pred_class])
predictions.append(letter_map[pred_class])
confidences.append(confidence)
return predictions, confidences

```

iii. RESTful APIs

```

from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy
from werkzeug.security import generate_password_hash, check_password_hash
from functools import wraps
import jwt
from gtts import gTTS
from datetime import datetime, timedelta

app = Flask(__name__)
db = SQLAlchemy(app)

# App Configuration
app.config.update(
    SECRET_KEY='your_secret_key',
    SQLALCHEMY_DATABASE_URI='sqlite:///devanagari.db',
    JWT_EXPIRATION_HOURS=24
)

# Database Models
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(255), nullable=False)

```

```

class Prediction(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    prediction = db.Column(db.String(10), nullable=False)
    confidence = db.Column(db.Float, nullable=False)

# Authentication decorator
def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        auth_header = request.headers.get('Authorization', '')
        if not auth_header or not auth_header.startswith('Bearer '):
            return jsonify({'status': 'error', 'message': 'Invalid Authorization header format'}), 401
        try:
            token = auth_header.split(' ')[1]
            data = jwt.decode(token, app.config['SECRET_KEY'], algorithms=['HS256'])
            current_user = User.query.get(data['user_id'])
            if not current_user:
                return jsonify({'status': 'error', 'message': 'User not found'}), 401
        except jwt.ExpiredSignatureError:
            return jsonify({'status': 'error', 'message': 'Token has expired'}), 401
        except jwt.InvalidTokenError:
            return jsonify({'status': 'error', 'message': 'Invalid token'}), 401
        return f(current_user, *args, **kwargs)
    return decorated

# Routes
@app.route('/api/signup', methods=['POST'])
def signup():
    data = request.get_json()
    username = data.get('username')
    email = data.get('email')
    password = data.get('password')
    if not username or not email or not password:

```

```

        return jsonify({'status': 'error', 'message': 'Missing required fields'}), 400
    if User.query.filter_by(email=email).first():
        return jsonify({'status': 'error', 'message': 'Email already exists'}), 400
    hashed_password = generate_password_hash(password)
    new_user = User(username=username, email=email, password=hashed_password)
    db.session.add(new_user)
    db.session.commit()
    return jsonify({'status': 'success', 'message': 'User created successfully'}), 201

@app.route('/api/signin', methods=['POST'])
def signin():
    data = request.get_json()
    email = data.get('email')
    password = data.get('password')
    if not email or not password:
        return jsonify({'status': 'error', 'message': 'Email and password are required'}), 400
    user = User.query.filter_by(email=email).first()
    if not user or not check_password_hash(user.password, password):
        return jsonify({'status': 'error', 'message': 'Invalid email or password'}), 401
    token = jwt.encode({
        'user_id': user.id,
        'exp': datetime.utcnow() +
timedelta(hours=app.config['JWT_EXPIRATION_HOURS'])
    }, app.config['SECRET_KEY'])
    return jsonify({'status': 'success', 'token': token}), 200

@app.route('/api/predict', methods=['POST'])
@token_required
def predict(current_user):
    if 'image' not in request.files:
        return jsonify({'status': 'error', 'message': 'No image file provided'}), 400
    image_file = request.files['image']
    image_data = image_file.read()
    segments = process_image(image_data)
    predictions, confidences = predict_segments(segments)

```

```
    return jsonify({'status': 'success', 'predictions': predictions, 'confidences': confidences}),  
    200
```

```
@app.route('/api/generate-audio', methods=['POST'])
```

```
@token_required
```

```
def generate_audio(current_user):
```

```
    data = request.get_json()
```

```
    text = data.get('text')
```

```
    if not text:
```

```
        return jsonify({'status': 'error', 'message': 'No text provided'}), 400
```

```
    tts = gTTS(text=text, lang='ne')
```

```
    audio_path = f"audio/{datetime.now().strftime('%Y%m%d_%H%M%S')}.mp3"
```

```
    tts.save(audio_path)
```

```
    return jsonify({'status': 'success', 'audio_url': audio_path}), 200
```

```
@app.route('/api/logout', methods=['POST'])
```

```
@token_required
```

```
def logout(current_user):
```

```
    return jsonify({'status': 'success', 'message': 'Logged out successfully'}), 200
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```